

Defect Prediction

By: Huy Nguyen and Edward Feng



Data Collection

- PyGithub/RepoMiner/pydriller
 - Gets all commits
 - Date
 - Commit message
 - Files
- Nature Language Processing libraries to extract keywords from commit messages (spaCy/NLTK)
 - Then check which ones include the keyword
- OR research SZZUnleashed
 - Have to install manually

Potential Parameters in the Formula

- Recency (Date committed) (Iteration 1)
- Frequency (How often the files are changed?) (Iteration 1)
- Message keywords
 - Very depending on developer preference so might not work/less weight
- Number of lines changed (Iteration 1)
- How many lines of code in the file? (Considered)

Calculating Weight

$$\text{Hotspot Score} = w1 \cdot R + w2 \cdot F + w3 \cdot M + w4 \cdot C + w5 \cdot L$$

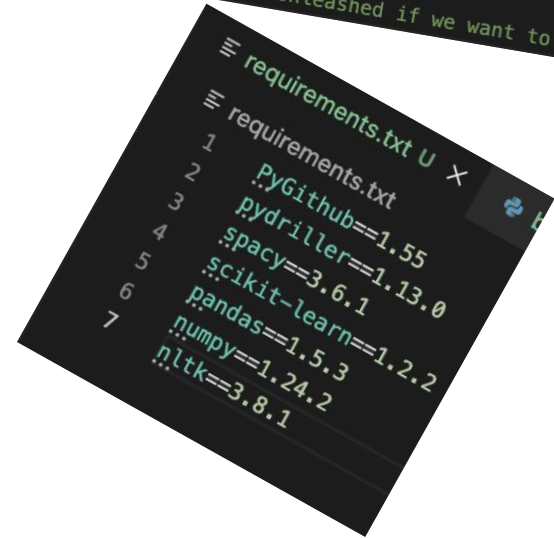
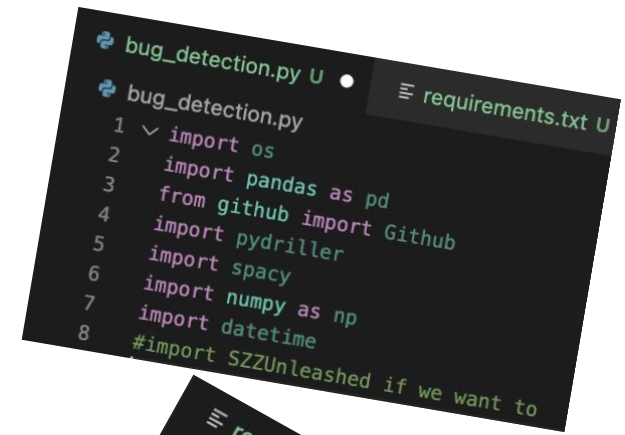
- R = Recency score (recent changes matter more)
- F = Frequency score (higher frequency means more unstable)
- M = Message keyword score (detects defect-related words)
- C = Number of lines changed (large changes indicate more risk)
- L = File size (larger files tend to be harder to maintain)

Weight can be human heuristic or determined through data science techniques such as PCA (sklearn+TfidfVectorizer).

Timeline (2 Sprints per Iteration)

- Iteration 1: Research and develop a prototype (demo) for calculating files hotspot for bugs of one Github repo
- Iteration 2: Review feedbacks and apply changes
- Iteration 3: Calculate files hotspot for bugs of multiple Github repo/given repo after implementing feedbacks

Files Structure



Questions

- What should our output be? (top 3? A score for every file?)
- Repominer version doesn't really work with others (Pydriller might be able to do the same thing as well). Is it required for us to use it?
- Does the example formula given in class work?

[illegible]